158.337
Database Development

Week 8

Procedural SQL

Cursors & Triggers

Objectives

Understanding stored procedures –

- Using cursors in stored procedures
  - Implicit
  - Explicit
- Triggers
- User defined exceptions

Implicit Cursors

So far, we have retrieved a single row of data in a PL/SQL block. Every time a SQL statement is executed an implicit cursor is declared.

The cursor is a work area in memory where the statement is processed that contains the results of the SQL statement.

SQL Cursor Attributes - checks the result of an SQL statement to see whether the statement affects any rows and how many.

%ROWCOUNT – number of rows affected

%FOUND – TRUE if at least one row is found

%NOTFOUND – TRUE if no rows are affected

%ISOPEN – TRUE if cursor is already open

Implicit Cursors with Cursor Attributes

```
1  DECLARE
2      prod_descript   product.p_descript%TYPE;
3      vend_name       vendor.v_name%TYPE;
4  BEGIN
5      SELECT p.p_descript prod_descript,v.v_name vend_name
6      INTO prod_descript,vend_name
7      FROM product p, vendor v
8      WHERE
9          p.v_code = v.v_code AND   p.p_price < 5;
10     dbms_output.put_line(SQL%rowcount);
11     IF
12         SQL%notfound
13     THEN
14         dbms_output.put_line('not found');
15     END IF;
16 END;
17 /
```

```
1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      UPDATE product
5          SET p_qoh = p_qoh + 10
6          WHERE p_code = '11QER/31';
7      dbms_output.put_line(SQL%rowcount);
8      IF
9          SQL%notfound
10     THEN
11         dbms_output.put_line('not found');
12     END IF;
13 END;
14 /
15 --rollback;
```

Implicit Cursors with Exceptions

Find the product that are not in-house (or are supplied by a vendor) and the product price is < $5

```
1  DECLARE
2      prod_descript   product.p_descript%TYPE;
3      vend_name       vendor.v_name%TYPE;
4  BEGIN
5      SELECT p.p_descript prod_descript,v.v_name vend_name
6      INTO prod_descript,vend_name
7      FROM product p, vendor v
8      WHERE
9          p.v_code = v.v_code AND   p.p_price < 5;
10         DBMS_OUTPUT.PUT_LINE (prod_descript || '--is provided by --' || vend_name);
11 EXCEPTION
12     when no_data_found then
13     dbms_output.put_line ('There is no such product');
14 END;
```

```
Rat-tail file, 1/8-in. fine--is provided by --Gomez Bros.

PL/SQL procedure successfully completed.
```

PL/SQL with Multiple Row Queries

But if any SQL query returns more than one row of data, we cannot use implicit cursors alone, and the PL/SQL statement gives error

ORA-01422 exact fetch returns more than requested number of rows

Exception

TOO_MANY_ROWS - A SELECT statement in a PL/SQL block retrieves more than one row.

```
SELECT p.p_descript prod_descript,v.v_name vend_name
INTO prod_descript,vend_name
FROM product p, vendor v
WHERE
    p.v_code = v.v_code AND   p.p_price < 10;
```

```
Error report -
ORA-01422: exact fetch returns more than requested number of rows
ORA-06512: at line 5
01422. 00000 -  "exact fetch returns more than requested number of rows"
*Cause:    The number specified in exact fetch is less than the rows returned.
*Action:   Rewrite the query or change number of rows requested
```

PL/SQL with Multiple Row Queries

So,we need to create a named cursor (or explicit cursor) programmatically so that it can be referenced for processing each row at a time.

Cursors introduce the ability to handle multiple rows of data.

Cursor-style processing involves retrieving data from the cursor one row at a time.

Current row is copied to PL/SQL variables

PL/SQL Processing with Cursors

For Oracle to process a SQL result, it needs to create an area of memory known as the context area; this will have the information needed to process the statement.

A cursor is a handle, or a pointer, to the context area. Through the cursor, a PL/SQL program can control the context area and what happens to it as the statement is processed.

Explicit Cursor

An explicit cursor is used when the PL/SQL uses an SQL query which returns more than one row.

Steps

1. Declaring the cursor – This initializes the cursor into memory.
2. Opening the cursor – The initialized cursor is opened, and memory is allotted.
3. Fetching the cursor – The declared and opened cursor can now retrieve data
4. Closing the cursor – The declared, opened, and fetched cursor must be closed to release the memory allocation

Declaring a Cursor

Declaring a cursor defies the cursor name and associates it with a SELECT statement

CURSOR c_cursor_name IS select statement;

Example:

```
DECLARE
    CURSOR  c_MyCursor IS
        Select * from customer where cus_areacode ='615';
```

Record Types – Table Based

A record type is a composite data structure, which means that it is composed of one or more elements.

PL/SQL supports three kinds of records: table-based, cursor-based and programmer-defined.

We will do only table-based records.

A table-based record is one whose structure is drawn from the list of columns in the table.

Record Types – Table Based

To define a table-based record, we use the %ROWTYPE attribute

Record_name Cursor_name%ROWTYPE;

Now, to reference columns within the table we use following syntax

Record_name.column_name

Making use of record types

vr_customer_name has structure similar to the row returned by the SELECT statement defined in the cursor.

```
SET SERVEROUTPUT ON;

DECLARE CURSOR c_mycursor IS
SELECT
   cus_lname,
   cus_fname,
   cus_initial
FROM  customer;

vr_customer_name c_mycursor%rowtype;
```

Cursor Attributes

To determine the result of a cursor operation when fetched or opened, we use the following attributes

Cursor_Name%NOTFOUND
Cursor_Name%FOUND
Cursor_Name%ROWCOUNT
Cursor_Name%ISOPEN

e.g.

```
EXIT WHEN Cursor_Name%NOTFOUND

IF Cursor_Name%FOUND THEN
    DBMS_OUTPUT.PUT_LINE ('Just fetched row' ||
    TO_CHAR(Cursor_Name%ROWCOUNT) || 'Customer code: ' || CUST_CODE;

IF  Cursor_Name%ISOPEN
    THEN CLOSE Cursor_Name
```

Cursors and Record Types

```
1  SET SERVEROUTPUT ON;
2  DECLARE CURSOR c_mycursor is
3  SELECT  cus_lname,  cus_fname,  cus_initial
4  FROM  customer;
5  vr_customer_name c_mycursor%rowtype;
6  BEGIN
7  OPEN c_mycursor;
8  LOOP
9  FETCH c_mycursor into vr_customer_name;
10 EXIT WHEN c_mycursor%NOTFOUND;
11 DBMS_OUTPUT.PUT_LINE (vr_customer_name.cus_fname|| ' '
12 || vr_customer_name.cus_initial||' '
13 || vr_customer_name.cus_lname);
14 END LOOP;
15 CLOSE c_mycursor;
16 END;
```

Script Output

Task completed in 0.032 seconds

```
Alfred A Ramas
Leona K Dunne
Kathy W Smith
Paul F Olowski
Myron  Orlando
Amy B O'Brian
James G Brown
George  Williams
Anne G Farriss
Olette K Smith
Sean  Connery


PL/SQL procedure successfully completed.
```

Fetching rows in a cursor

The LOOP statement is used as there are more than one rows returned.

Next, we FETCH each row until NOTFOUND condition.

CLOSE the cursor

```
1  SET SERVEROUTPUT ON;
2  DECLARE CURSOR c_mycursor is
3  SELECT  cus_lname,  cus_fname,  cus_initial
4  FROM  customer;
5  vr_customer_name c_mycursor%rowtype;
6  BEGIN
7  OPEN c_mycursor;
8  LOOP
9  FETCH c_mycursor into vr_customer_name;
10 EXIT WHEN c_mycursor%NOTFOUND;
11 DBMS_OUTPUT.PUT_LINE (vr_customer_name.cus_fname|| ' '
12 || vr_customer_name.cus_initial||' '
13 || vr_customer_name.cus_lname);
14 END LOOP;
15 CLOSE c_mycursor;
16 END;
```

Step	Step Activity	Activity Description
1	DECLARE	Creates a named cursor identified by a SELECT statement. The SELECT statement does not include an INTO clause. Values in the cursor are moved to PL/SQL variables with the FETCH step.
2	OPEN	Processes the query and creates the active set of rows available in the cursor.
3	FETCH	Retrieves a row from the cursor into block variables. Each consecutive FETCH issued will retrieve the next row in the cursor until all rows have been retrieved.
4	CLOSE	Clears the active set of rows and frees the memory area used for the cursor.

Cursor Attributes - Cursor_Name%FOUND

```
DECLARE CURSOR c_mycursor is
    SELECT  cus_lname,  cus_fname,  cus_initial
    FROM  customer;
vr_customer_name c_mycursor%rowtype;
BEGIN
    OPEN c_mycursor;
          LOOP
          FETCH c_mycursor into vr_customer_name;
          IF c_mycursor%FOUND THEN DBMS_OUTPUT.PUT_LINE
          (vr_customer_name.cus_fname||' '||
          vr_customer_name.cus_initial||' '||
          vr_customer_name.cus_lname);
    ELSE
          EXIT;
    END IF;
END LOOP;
CLOSE c_mycursor;
END;
```

Cursor Attributes - Cursor_Name%ROWCOUNT

```
DECLARE CURSOR c_mycursor is
    SELECT  cus_lname,  cus_fname,  cus_initial
    FROM  customer;
vr_customer_name c_mycursor%rowtype;
BEGIN
    OPEN c_mycursor;
          LOOP
          FETCH c_mycursor into vr_customer_name;
          IF c_mycursor%FOUND THEN DBMS_OUTPUT.PUT_LINE(
    'Just fetched row '|| to_CHAR(c_mycursor%ROWCOUNT)|| ' ' ||
    vr_customer_name.cus_fname|| ' ' ||
          vr_customer_name.cus_initial||' ' ||
          vr_customer_name.cus_lname);
    ELSE
          EXIT;
    END IF;
END LOOP;
CLOSE c_mycursor;
END;
```

Worksheet | Query Builder

```
1  DECLARE CURSOR c_mycursor is
2      SELECT  cus_lname,  cus_fname,  cus_initial
3      FROM  customer;
4  vr_customer_name c_mycursor%rowtype;
5  BEGIN
6      OPEN c_mycursor;
7          LOOP
8          FETCH c_mycursor into vr_customer_name;
9          IF c_mycursor%FOUND THEN
10         DBMS_OUTPUT.PUT_LINE( 'Just fetched row '|| to_CHAR(c_mycursor%ROWCOUNT)
11         || ' ' ||  vr_customer_name.cus_fname|| ' ' ||  vr_customer_name.cus_initial
12         ||' ' ||    vr_customer_name.cus_lname);
13     ELSE
14         EXIT;
15     END IF;
16 END LOOP;
17 CLOSE c_mycursor;
18 END;
```

Task completed in 0.038 seconds

```
Just fetched row 1 Alfred A Ramas
Just fetched row 2 Leona K Dunne
Just fetched row 3 Kathy W Smith
Just fetched row 4 Paul F Olowski
Just fetched row 5 Myron  Orlando
Just fetched row 6 Amy B O'Brian
Just fetched row 7 James G Brown
Just fetched row 8 George  Williams
Just fetched row 9 Anne G Farriss
Just fetched row 10 Olette K Smith
Just fetched row 11 Sean  Connery


PL/SQL procedure successfully completed.
```

CURSOR with RECURSIVE QUERY

```
1  DECLARE CURSOR c_mycursor is
2      SELECT E.EMP_NUM as EMPNUM, E.EMP_LNAME as EMPLNAME,
3      E.EMP_MGR as MGRNUM, M.EMP_LNAME as MGRNAME
4  FROM EMP E, EMP M
5  WHERE E.EMP_MGR=M.EMP_NUM;
6
7  vr_emp_mgr c_mycursor%rowtype;
8
9  BEGIN
10     OPEN c_mycursor;
11         LOOP
12         FETCH c_mycursor into vr_emp_mgr;
13         IF c_mycursor%FOUND THEN DBMS_OUTPUT.PUT_LINE(
14         'Just fetched row '|| to_CHAR(c_mycursor%ROWCOUNT)|| ' ' ||  vr_emp_mgr.empnum|| ' ' ||
15         vr_emp_mgr.emplname||' ' ||     vr_emp_mgr.mgrnum||' ' ||   vr_emp_mgr.mgrname);
16     ELSE
17         EXIT;
18     END IF;
19 END LOOP;
20 CLOSE c_mycursor;
21 EXCEPTION
22 WHEN OTHERS
23 THEN
24 IF c_mycursor%ISOPEN
25 THEN CLOSE c_mycursor;
26 END IF;
27 END;
```

	COLUMN_NAME
1	EMP_NUM
2	EMP_TITLE
3	EMP_LNAME
4	EMP_FNAME
5	EMP_INITIAL
6	EMP_DOB
7	EMP_HIRE_DATE
8	EMP_AREACODE
9	EMP_PHONE
10	EMP_MGR

Cursor Attributes - Cursor_Name%ISOPEN

```
1  DECLARE CURSOR c_mycursor is
2      SELECT  cus_lname,  cus_fname,  cus_initial
3      FROM  customer;
4  vr_customer_name c_mycursor%rowtype;
5  BEGIN
6      OPEN c_mycursor;
7          LOOP
8          FETCH c_mycursor into vr_customer_name;
9          IF c_mycursor%FOUND THEN DBMS_OUTPUT.PUT_LINE( 'Just fetched row '||
10         to_CHAR(c_mycursor%ROWCOUNT)|| ' ' ||  vr_customer_name.cus_fname|| ' '
11         ||  vr_customer_name.cus_initial||' ' ||    vr_customer_name.cus_lname);
12     ELSE
13         EXIT;
14     END IF;
15 END LOOP;
16 EXCEPTION
17 WHEN OTHERS THEN
18 DBMS_OUTPUT.PUT_LINE( 'Some other errors have occured');
19 IF c_mycursor%ISOPEN
20 THEN CLOSE c_mycursor;
21 END IF;
22 END;
```

Good Practice to use Exception to close the cursor properly.

Cursor - Examples

```
CREATE TABLE STUDENT (
STUD_NUM        NUMBER PRIMARY KEY,
STUD_LNAME      VARCHAR(20),
STUD_FNAME      VARCHAR(20),
STUD_AGE NUMBER);
```

is123456Conn | STUDENT

Columns | Data | Constraints | Grants | Statis

Actions...

Column Name	Data Type
STUD_NUM	NUMBER
STUD_LNAME	VARCHAR2(20 BYTE)
STUD_FNAME	VARCHAR2(20 BYTE)
STUD_AGE	NUMBER

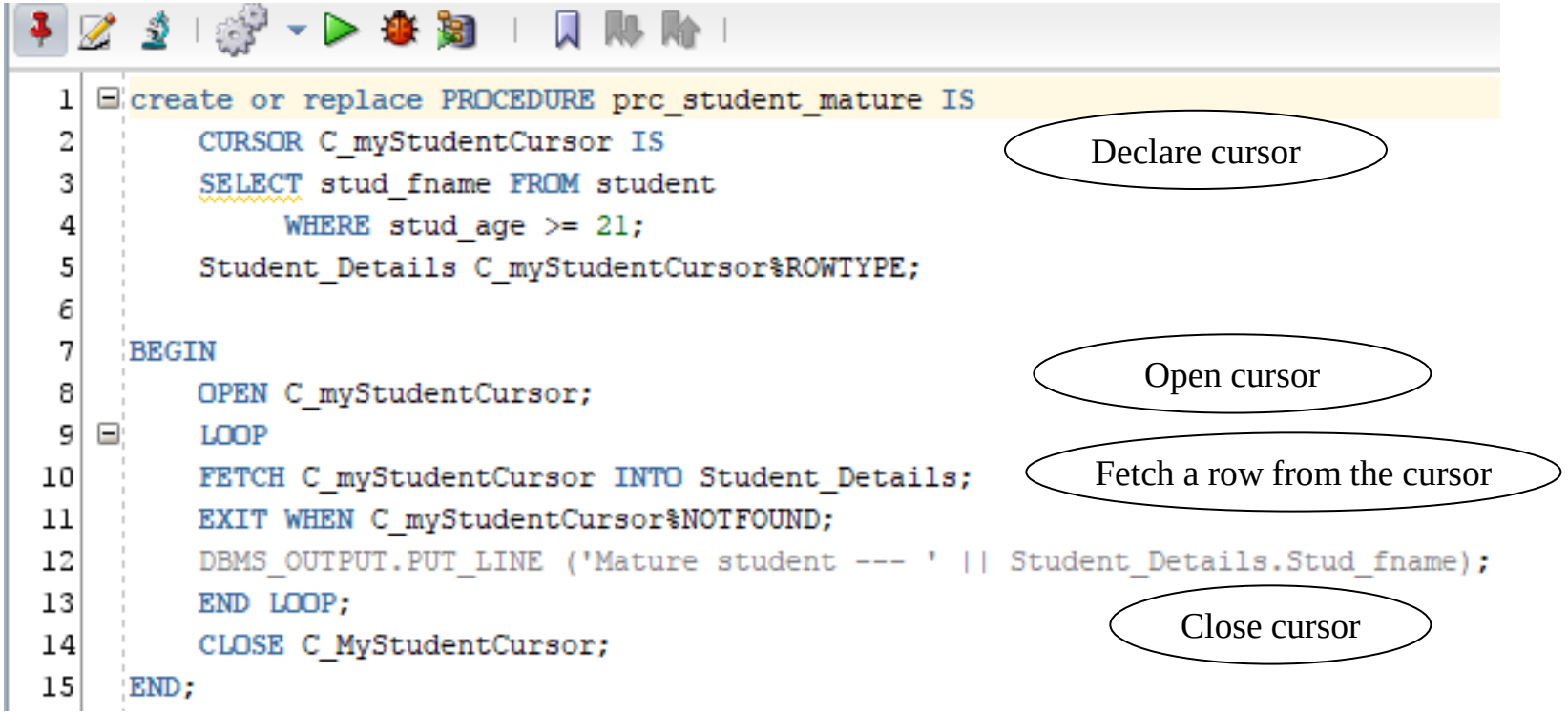
```
INSERT INTO STUDENT VALUES(2001 ,'McPherson' ,'Anne', 24);
INSERT INTO STUDENT VALUES(2002 ,'Ortega'    ,'Juan', 20);
INSERT INTO STUDENT VALUES(2003 ,'Kowalski'  ,'Jan', 18);
INSERT INTO STUDENT VALUES(2004 ,'Smith'  ,'Jane', 38);
INSERT INTO STUDENT VALUES(2005 ,'Kane'  ,'Sam', 48);
```

is123456Conn | STUDENT

Columns | Data | Constraints | Grants | Statistics | Triggers | Flashback | Dependenci

Sort... | Filter:

	STUD_NUM	STUD_LNAME	STUD_FNAME	STUD_AGE
1	2001	McPherson	Anne	24
2	2002	Ortega	Juan	21
3	2003	Kowalski	Jan	18
4	2004	Smith	Jane	38
5	2005	Kane	Sam	48



```
1  create or replace PROCEDURE prc_student_mature IS
2      CURSOR C_myStudentCursor IS
3      SELECT stud_fname FROM student
4          WHERE stud_age >= 21;
5      Student_Details C_myStudentCursor%ROWTYPE;
6
7  BEGIN
8      OPEN C_myStudentCursor;
9      LOOP
10     FETCH C_myStudentCursor INTO Student_Details;
11     EXIT WHEN C_myStudentCursor%NOTFOUND;
12     DBMS_OUTPUT.PUT_LINE ('Mature student --- ' || Student_Details.Stud_fname);
13     END LOOP;
14     CLOSE C_MyStudentCursor;
15 END;
```



```
Procedure PRC_STUDENT_MATURE compiled

Mature student --- Anne
Mature student --- Jane
Mature student --- Sam


PL/SQL procedure successfully completed.
```

CURSORS – FOR Loops

For Loops uses simplified syntax.

The process of opening, fetching and closing is handled implicitly.
Here we just define a variable to loop thru…

```
SET SERVEROUTPUT ON;
DECLARE CURSOR c_mycursor is
SELECT  cus_lname,  cus_fname,  cus_initial FROM  customer;
BEGIN
        FOR cust IN c_mycursor
        LOOP
                DBMS_OUTPUT.PUT_LINE (cust.cus_fname|| ' ' || cust.cus_initial||' '
                || cust.cus_lname);
        END LOOP;
END;
```

FOR Loop Examples

```
1  DECLARE CURSOR c_mycursor is
2      SELECT  cus_lname,  cus_fname,  cus_initial
3      FROM  customer;
4  BEGIN
5      FOR c IN c_mycursor
6      LOOP
7          update customer
8          set cus_initial = 'X'
9          where cus_initial is null;
10     END LOOP;
11     commit;
12 END;
```

Note the declaration of c_mycursor (for multiple rows) and c (individual row).

Also, note the use of commit statement.

```
1  DECLARE CURSOR c_mycursor is
2       SELECT P.P_DESCRIPT descrip, V.V_NAME vname
3  FROM   PRODUCT P, VENDOR V
4  WHERE  P.V_CODE = V.V_CODE;
5  BEGIN
6      FOR c IN c_mycursor
7      LOOP
8          dbms_output.put_line(c.descrip || ' ---- ' || c.vname);
9      END LOOP;
10 END;
11
```

Note the use of alias when using 2 tables.

Procedure, Cursor, Grouping, For

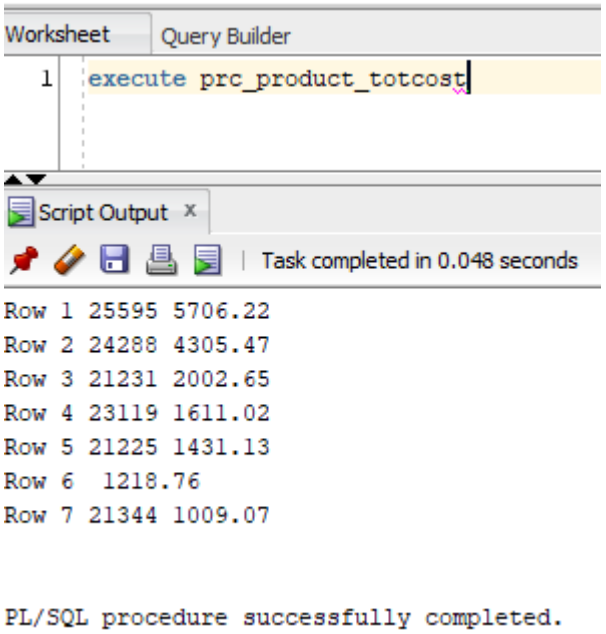
```
1  SELECT V_CODE, SUM(P_QOH * P_PRICE) AS TOTCOST
2  FROM   PRODUCT
3  GROUP  BY V_CODE
4  HAVING (SUM(P_QOH * P_PRICE)>500)
5  ORDER  BY SUM(P_QOH * P_PRICE) DESC
```

```
   V_CODE    TOTCOST
--------- ----------
     25595    5706.22
     24288    4305.47
     21231    2002.65
     23119    1611.02
     21225    1431.13
              1218.76
     21344    1009.07

7 rows selected.
```

Procedure, Cursor, Grouping, For

```
1  CREATE OR REPLACE PROCEDURE PRC_PRODUCT_TOTCOST IS
2      CURSOR c_mycursor IS
3      SELECT
4          v_code, SUM(p_qoh * p_price) AS TOTCOST
5      FROM product
6      GROUP BY v_code
7      HAVING ( SUM(p_qoh * p_price) > 500 )
8      ORDER BY SUM(p_qoh * p_price) desc;
9
10 BEGIN
11     FOR vendor IN c_mycursor
12     LOOP
13         DBMS_OUTPUT.PUT_LINE('Row '|| c_mycursor%rowcount || ' '
14                     || vendor.v_code || ' ' || vendor.totcost);
15     END LOOP;
16 end;
```



Triggers

Procedural SQL code automatically invoked by RDBMS on data manipulation event

Trigger definition:

- Triggering timing: BEFORE or AFTER
- Triggering event: INSERT, UPDATE, DELETE
- Triggering level:
  - Statement-level trigger
  - Row-level trigger
- Triggering action

DROP TRIGGER trigger_name

Triggers

A trigger is a named PL/SQL block stored in a database and executed implicitly when a triggering event occurs. The act of executing a trigger is called firing the trigger.

A triggering event can be one of the following:

DML Statement – INSERT, UPDATE or DELETE

DDL Statement – CREATE or ALTER

System event – Startup or Shutdown of the database

User event – logon and logoff

Triggers

Procedural SQL code automatically invoked by RDBMS on data manipulation event

Trigger definition:

- Triggering timing: BEFORE or AFTER
- Triggering event: INSERT, UPDATE, DELETE
- Triggering level:
  - **Statement-level trigger** -- indicates firing the trigger only once for the event, regardless of the number of rows affected by the DML statement
  - **Row-level trigger** -- indicates firing the trigger code for each row affected in the DML statements

Triggers – General Syntax

```
CREATE TRIGGER trigger_name
    {BEFORE|AFTER} Triggering_event ON table_name
FOR EACH ROW
DECLARE
    declaration statements
BEGIN
    executable statements
Exception
    exception handling statements
END;

DROP TRIGGER trigger_name
```

Triggers

Triggers can be used to enforce constraints that cannot be enforced at the DBMS design and implementation levels.

Triggers add functionality by automating critical actions and providing appropriate warnings/suggestions for remedial action.

Triggers can be used to update table values, insert records in tables and call other procedures.

If a table or view is dropped, any associated DML triggers will automatically be deleted.

Triggers

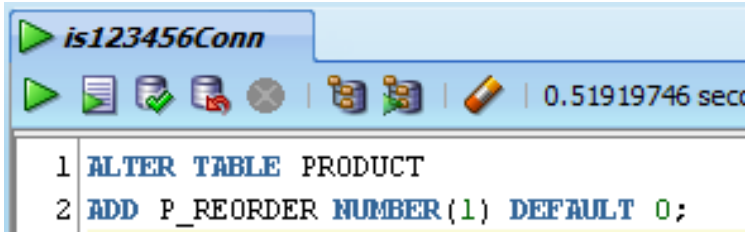
Example 1 - Statement Level Trigger – performs an UPDATE of all rows in the PRODUCT table even if triggering updates just one row. Here, you omit the keywords “FOR EACH ROW”. This is the default case and the trigger is executed once.

Example 2 – Row Level Trigger – ensures more data integrity. A copy of the row is updated before the original. This requires the use of “FOR EACH ROW”. So, if you update 10 rows, this trigger executes 10 times.



Triggers

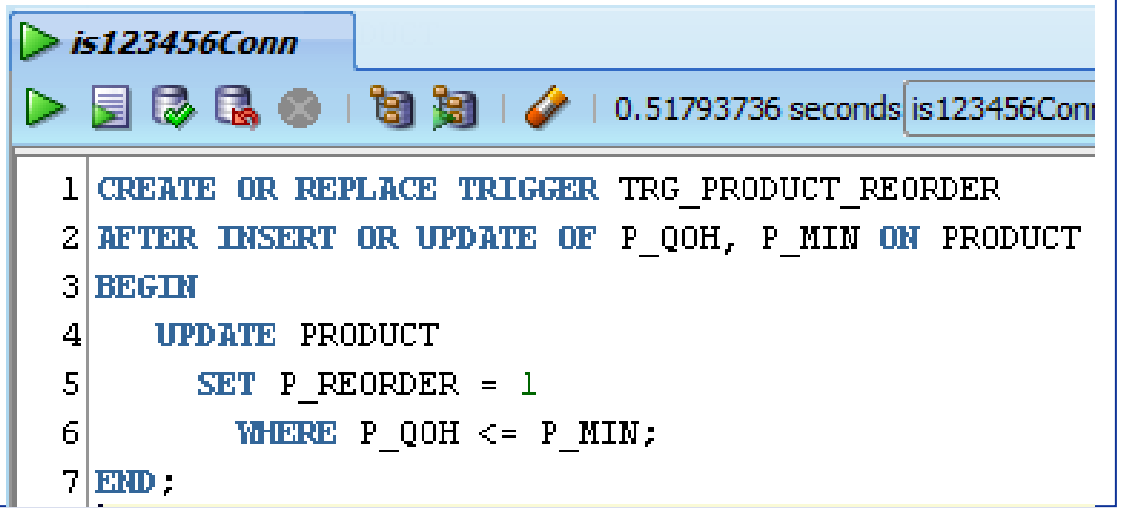
Using one book example



The triggering action occurs between the BEGIN and the END statements.

Example 1

Statement Level



Trigger Action

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_REORDER
23114-AA	Sledge hammer, 12 lb.	02/01/12	8	5	14.4	.05		0
54778-2T	Rat-tail file, 1/8-in. fine	15/12/11	43	20	4.99	.05	21344	0

Change the quantity level of 23114-AA so that P_QOH < P_MIN

```
1  UPDATE product
2  SET
3      p_min = 10
4  WHERE
5      p_code = '23114-AA';
```

The trigger TRG_PRODUCT_REORDER will get to work

```
1  SELECT  P_CODE, P_REORDER FROM PRODUCT;
```

P_CODE	P_REORDER
23114-AA	1
54778-2T	0

Now change the quantity of 23114-AA so that P_QOH > P_MIN

```
1  UPDATE product
2  SET
3      p_min = 5
4  WHERE
5      p_code = '23114-AA';
```

TRG_PRODUCT_REORDER has not reset the P_REORDER to 0

Let us try a row level trigger

```
1  CREATE OR REPLACE TRIGGER trg_product_reorder BEFORE
2      INSERT OR UPDATE OF p_qoh, p_min ON product
3      FOR EACH ROW
4  BEGIN
5      IF :new.p_qoh <= :new.p_min THEN
6          :new.p_reorder := 1;
7      ELSE
8          :new.p_reorder := 0;
9      END IF;
10 END;
```

Here :NEW refers to updated value (while :OLD will refer to the original value)

```
1  update product
2  set p_min = 5
3  where p_code = '23114-AA';
```

:NEW Attribute

The basic computing tenet is - all changes are first in primary memory, then to permanent memory (disk).

The DBMS makes two copies of every row being changed.

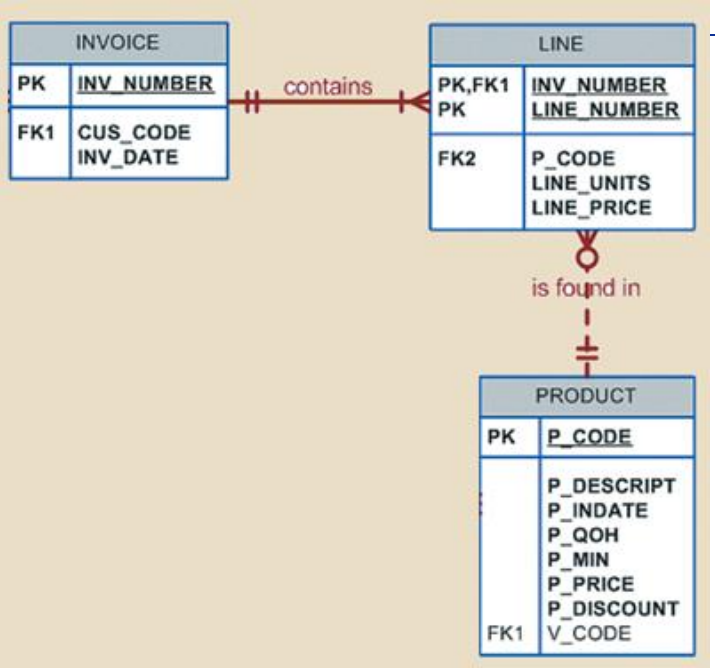
- The first copy contains original (:OLD) values before the changes.
- The second copy contains the changed (:NEW) values after changes are made by INSERT, DELETE or UPDATE

Also note the use of BEFORE and AFTER.

- BEFORE means before changes are permanently saved to disk, but after the changes are made in memory.
- AFTER means the subsequent action when the INSERT, UPDATE or DELETE operation has been executed.

Trigger

Example 3 : Update Product quantity when a new line is added to invoice.



Start Page | it123456Conn | PRODUCT | 1.37600005 seconds

Worksheet | Query Builder

```
1  create or replace trigger trg_line_add_prod
2  after insert on line
3  for each row
4  begin
5  update product
6  set p_qoh = p_qoh - :new.line_units
7  where
8  product.p_code = :new.p_code;
9  end;
10 /
```

TRIGGER statements

ALTER statement - Used to disable/enable a trigger

ALTER TRIGGER *trigger_name* DISABLE|ENABLE;

ALTER TABLE *table_name* DISABLE|ENABLE ALL TRIGGERS;

Although triggers are independent objects, they are associated with database tables. When you delete a table, all trigger objects are deleted with it. But you can delete a trigger without deleting the table by using the DROP TRIGGER command.

DROP TRIGGER *trigger_name*;

Exception Handling in Triggers

Triggers can often be used to enforce constraints. The WHEN clause or body of the trigger can check for the violation of certain conditions and signal an error accordingly using the Oracle built-in function RAISE_APPLICATION_ERROR.

The action that activated the trigger (INSERT, UPDATE or DELETE) would be aborted.

Example – Student enrolled in University must have

Age > 16

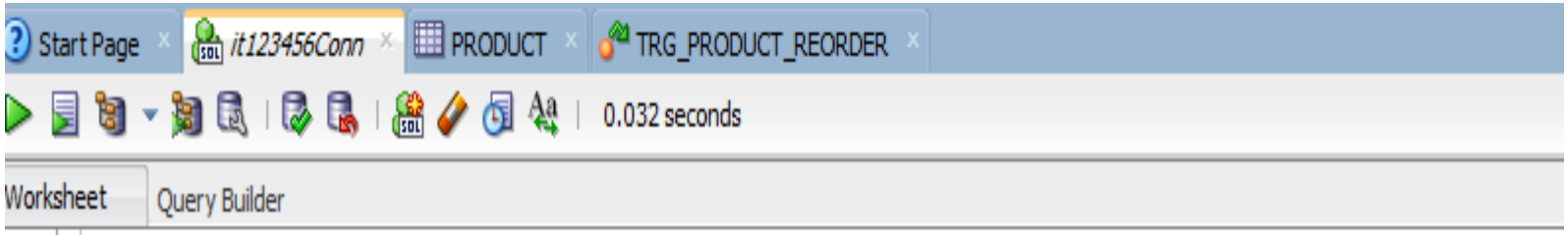
RAISE_APPLICATION_ERROR

RAISE_APPLICATION_ERROR lets you issue user-defined ORA-error messages from stored subprograms.

To call RAISE_APPLICATION_ERROR, use the syntax

```
raise_application_error(error_number, message);
```

where error_number is a negative integer in the range -20000 .. -20999 and message is a character string up to 2048 bytes long.

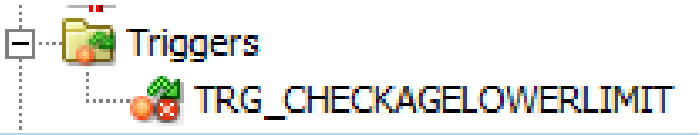



```
1  create or replace trigger trg_product_order
2  before update of p_qoh
3  on product
4  for each row
5  declare
6      qoh_below_min_exception exception;
7  begin
8      if (:new.p_qoh < :new.p_min) then
9
10         DBMS_OUTPUT.PUT_LINE ('Hey!!!');
11         raise qoh_below_min_exception;
12     end if;
13 exception
14
15 when qoh_below_min_exception then
16     raise_application_error(-20001,'The qoh will be less than min. First re-order and then place ordwer');
17 end;
18 /
```

```
INSERT INTO LINE VALUES(1109,4,'SM-18277',100,14.99)
Error report -
ORA-20001: The qoh will be less than min. First re-order and then place order
ORA-06512: at "IT337002.TRG_PRODUCT_ORDER", line 12
ORA-04088: error during execution of trigger 'IT337002.TRG_PRODUCT_ORDER'
ORA-06512: at "IT337002.TRG_LINE_ADD_PROD", line 2
ORA-04088: error during execution of trigger 'IT337002.TRG_LINE_ADD_PROD'
```

is123456Conn | TRG_CHECKAGELOWERLIMIT

0.6597755 seconds




```
1  create or replace trigger trg_checkagelowerlimit
2  before insert or update of stud_age
3  on student
4  for each row
5  declare
6    age_lower_than_18_error exception;
7  begin
8    if (:new.stud_age < 16) then
9      DBMS_OUTPUT.PUT_LINE ('Hey!!! Student age cannot be less than 16, therefore cannot update');
10     raise age_lower_than_18_error;
11   end if;
12
13 exception
14  when age_lower_than_18_error then
15   raise_application_error(-20001,'cannot insert or update');
16 end;
```

	STUD_NUM	STUD_LNAME	STUD_FNAME	STUD_AGE
1	2000	McPherson	Anne	24
2	2001	Ortega	Juan	20
3	2002	Kowalski	Jan	18
4	2003	Smith	Jane	38
5	2004	Kane	Sam	48

Trigger Limitations

1. Triggers cannot issue transaction control statements like
   - COMMIT
   - ROLLBACK
   - SAVEPOINT
2. Triggers cannot handle some datatypes like
   - LONG
   - LOB,
   - OBJECT
3. Mutating table issue: A table is being modified by a DML statement and at the same time the trigger attempts to query this same table.

Mutating Trigger Examples

- On insert triggers have no :OLD values
- On delete triggers have no :NEW values
- A trigger can cause other events to execute triggers
- A trigger can not change a table that it has read from
- If more than one trigger is defined on an event, the order in which they fire is not defined
- Re-visit your trigger design

Mapping the flow of triggers

Note that one statement can end up firing off a number of triggers.

Multiple triggers can be attached to a single table and a trigger can perform DML operations that could affect other tables and fire additional triggers leading to table mutating errors…

Therefore, it is important to make our code clear and maintainable.

Summary

Procedural Language SQL (PL/SQL) can be used to create triggers, stored procedures and PL/SQL functions.

A stored procedure is a named collection of SQL statements.

When SQL statements are designed to return more than one value inside the PL/SQL code, a cursor is needed.

Triggers are automatically fired when some particular database event to which this trigger is associated with occurs. This is quite a change from procedures and functions which are explicitly called by the program code.

Next week

We will learn query capabilities via LINQ - Language Integrated Query

The textbook does not cover this, but there are plenty of online resources available. In the meantime download LinqPad5 from https://www.linqpad.net/

You will find many sample code and examples on LINQ with different .NET languages here.

https://www.linqpad.net/Download.aspx

Download LINQPad

Download LINQPad 7 — for **.NET 6 / .NET 5 / .NET Core 3.1**
Release Notes

Download LINQPad 5 — for .NET Framework **4.6/4.7/4.8**
Release Notes

- Super-lightweight setup
- Standard edition **free** (no expiry)
- X64 + X86 + ARM64 (Windows Desktop)

LINQPad 7 and LINQPad 5 run side-by-side.

More Download Options

Not ready for LINQPad 7?	**Get LINQPad 6**
Want the very latest features?	Get the latest LINQPad 7 beta Get the latest LINQPad 5 beta
Hate installers?	Download xcopy-deploy build (LINQPad 7) Download xcopy-deploy build (LINQPad 5)
Running massive queries in LINQPad 5?	Download LINQPad 5 for X64 (installer) Download LINQPad 5 for X64 (xcopy-deploy)

